

QSS REST API v1 — Developer Guide (For Exhibitors)

This guide provides complete documentation for the QSS REST API v1, including authentication, endpoints, request/response formats, and code examples.

Overview

The QSS REST API v1 allows exhibitors to programmatically access registrant data for their assigned events. The API follows RESTful conventions and is inspired by the [Swoogo API](#) design.

Base URL: `https://qss.qestsoln.com/api/v1`

Authentication

Step 1: Obtain Your API Credentials

Contact your QSS administrator to obtain your **API Key** and **API Secret**. These are long strings that look like:

```
api_key:    qss_6cfb54cbf59ec6a14f9553a3af989e70
api_secret: 23d4551385b837db667cf8e65108d51507d0f45a
```

Step 2: Exchange Credentials for a Bearer Token

Before calling any data endpoint, you must exchange your API Key and Secret for a temporary Bearer Token (JWT). Tokens expire after **30 minutes**.

Request:

```
POST /api/v1/oauth/token HTTP/1.1
Host: qss.qestsoln.com
Authorization: Basic base64(api_key:api_secret)
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials
```

How to construct the Basic Auth header:

Base64-encode the string `api_key:api_secret` (colon-separated):

```
# Example using command line
echo -n "qss_6cfb54cbf59ec6a14f9553a3af989e70:23d4551385b837db667cf8e65108d5150" | xxd -ps | tr -dc 'A-Z' | fold -w 32 | paste -sd ':' - | sed 's/:$//'
# Result: cXNzXzZjZmI1NGNiZjU5ZWMyTE0Zjk1NTNhM2FmOTg5ZTcwOjJlZDQ1NTEzODViODM3
```

Response (200):

```
{
  "access_token": "eyJ0eXAiOiJKV1QiLCJKBV1",
  "expires_at": "2026-07-02 00:52:33",
  "type": "bearer"
}
```

cURL Example:

```
curl -X POST https://qss.questsoln.com/api/v1/oauth/token \
-H "Authorization: Basic *** \
-H "Content-Type: application/x-www-form-urlencoded" \
-d "grant_type=client_credentials"
```

Step 3: Use the Bearer Token

Include the token in the `Authorization` header for all subsequent requests:

Authorization: Bearer eyJ0eXAiOiJKV1QiLCJ...

Endpoints

1. List Registrants

Retrieve a paginated list of registrants for your event.

```
GET /api/v1/registrants
```

Query Parameters:

Parameter	Required	Default	Description
event_id	Yes	—	Your event ID (GUID format, e.g. AB6E9834-5752-2A1A-2CF9-58391339841B)
page	No	1	Page number (1-indexed)
per-page	No	20	Items per page (max: 100)
search	No	—	Search keyword (matches login_name or unique_id)
fields	No	—	Comma-separated field names to return only specific fields

Response (200):

```
{
  "items": [
    {
      "id": 959,
      "unique_id": "DB63B506-3536-B771-018B-25F1748AF252",
      "event_id": "AB6E9834-5752-2A1A-2CF9-58391339841B",
      "login_name": "jerry.bi",
      "type": "",
      "checkin_status": 0,
      "checkin_time": null,
      "event_name": "PSDay Malaysia",
      "zone": null,
      "table_no": null
    }
  ],
}
```

```
    "_meta": {
      "totalCount": 991,
      "pageCount": 50,
      "currentPage": 1,
      "perPage": 20
    }
  }
```

cURL Example:

```
curl https://qss.qestsoln.com/api/v1/registrants?event_id=AB6E9834-5752-2A1A-2C
-H "Authorization: Bearer YOUR_TOKEN"
```

2. Get a Single Registrant

Retrieve complete details for a specific registrant by their `unique_id` .

```
GET /api/v1/registrants/{unique_id}
```

Path Parameters:

Parameter	Description
<code>unique_id</code>	The registrant's unique identifier (GUID format)

Query Parameters:

Parameter	Required	Default	Description
<code>event_id</code>	Yes	—	Your event ID
<code>fields</code>	No	—	Comma-separated field names (see Available Fields below)

Response (200):

```
{
  "status": "success",
  "message": "OK",
  "data": {
    "id": 959,
    "unique_id": "DB63B506-3536-B771-018B-25F1748AF252",
    "event_id": "AB6E9834-5752-2A1A-2CF9-58391339841B",
    "event_name": "PSDay Malaysia",
    "login_name": "jerry.bi",
    "type": "",
    "status": 1,
    "checkin_status": 0,
    "checkin_time": null,
    "zone": null,
    "table_no": null,
    "enable_track": 1,
    "data_fields": {
      "email": "jerry@example.com",
      "first_name": "Jerry",
      "last_name": "Bi",
      "organisation": "QSS",
      "login_name": "jerry.bi",
      "password": ""
    },
    "scan_records": []
  }
}
```

Available Fields (for `fields` parameter):

Standard fields: `id`, `unique_id`, `event_id`, `event_name`, `login_name`, `type`, `status`, `checkin_status`, `checkin_time`, `zone`, `table_no`, `enable_track`, `data_fields`, `scan_records`

Custom data fields: Any key from `data_fields` (e.g., `first_name`, `last_name`, `email`)

Example with field filtering:

```
curl "https://qss.qestsoln.com/api/v1/registrants/DB63B506-3536-B771-018B-25F1748AF252" \
-H "Authorization: Bearer YOUR_TOKEN"
```

Response with filtering returns only requested fields:

```
{
  "status": "success",
  "message": "OK",
  "data": {
    "id": 959,
    "unique_id": "DB63B506-3536-B771-018B-25F1748AF252",
    "event_name": "PSDay Malaysia",
    "first_name": "Jerry",
    "last_name": "Bi"
  }
}
```

Error Responses

All errors use a consistent JSON format:

```
{
  "name": "Error Name",
  "message": "Human-readable description",
  "code": 0,
  "status": 404
}
```

HTTP Status Codes

Status	Name	Description
400	Bad Request	Missing required parameter (e.g., <code>event_id</code>)
401	Unauthorized	Missing/invalid token, or invalid API credentials
403	Forbidden	Attempting to access data from an event you are not assigned to
404	Not Found	Registrant not found for the given <code>unique_id</code> and <code>event_id</code>
405	Method Not Allowed	Using a non-POST method on the OAuth endpoint

Status	Name	Description
429	Too Many Requests	Rate limit exceeded (2,000 requests / 10 minutes)
500	Internal Server Error	Unexpected server error

Rate Limiting

Each exhibitor is limited to **2,000 requests per 10-minute window**. Rate limit information is included in response headers:

Header	Description
X-Rate-Limit-Limit	Maximum requests per window (2000)
X-Rate-Limit-Remaining	Remaining requests in current window
X-Rate-Limit-Reset	Seconds until the window resets

When the limit is exceeded, the API returns `429 Too Many Requests` .

Data Isolation

Each exhibitor is assigned to one event. You can **only** query data for the event assigned to your account. Attempting to query a different `event_id` will return `403 Forbidden` .

To find your `event_id` , check the response of any successful API call — the `event_id` field in the data indicates your assigned event.

CORS Support

The API supports Cross-Origin Resource Sharing (CORS) for browser-based applications:

- **Allowed Origins:** `*` (all origins)
- **Allowed Methods:** `GET` , `POST` , `PUT` , `DELETE` , `OPTIONS`
- **Allowed Headers:** `Authorization` , `Content-Type` , `Accept` , `Origin` , `X-Requested-With`

Preflight `OPTIONS` requests return `204 No Content` .

Complete Code Examples

PHP

```
<?php
$apiKey = 'qss_6cfb54cbf59ec6a14f9553a3af989e70';
$apiSecret = '23d4551385b837db667cf8e65108d51507d0f45a';
$baseUrl = 'https://qss.qestsoln.com/api/v1';

// Step 1: Get Bearer Token
$ch = curl_init($baseUrl . '/oauth/token');
curl_setopt_array($ch, [
    CURLOPT_POST => true,
    CURLOPT_RETURNTRANSFER => true,
    CURLOPT_HTTPHEADER => [
        'Authorization: Basic ' . base64_encode("$apiKey:$apiSecret"),
        'Content-Type: application/x-www-form-urlencoded',
    ],
    CURLOPT_POSTFIELDS => 'grant_type=client_credentials',
]);
$res = json_decode(curl_exec($ch), true);
curl_close($ch);

$token = $res['access_token'];

// Step 2: List registrants
$eventId = 'AB6E9834-5752-2A1A-2CF9-58391339841B';
$ch = curl_init($baseUrl . '/registrants?event_id=' . urlencode($eventId) . '&');
curl_setopt_array($ch, [
    CURLOPT_RETURNTRANSFER => true,
    CURLOPT_HTTPHEADER => [
        'Authorization: Bearer ' . $token,
    ],
]);
```



```

]);
$resp = json_decode(curl_exec($ch), true);
curl_close($ch);

print_r($resp);

```

Python

```

import requests

API_KEY = 'qss_6cfb54cbf59ec6a14f9553a3af989e70'
API_SECRET = '23d4551385b837db667cf8e65108d51507d0f45a'
BASE_URL = 'https://qss.qestsoln.com/api/v1'
EVENT_ID = 'AB6E9834-5752-2A1A-2CF9-58391339841B'

# Step 1: Get Bearer Token
resp = requests.post(f'{BASE_URL}/oauth/token',
    headers={'Authorization': f'Basic ***'}, data={'grant_type': 'client_credentials'})
token = resp.json()['access_token']

# Step 2: List registrants
resp = requests.get(f'{BASE_URL}/registrants',
    headers={'Authorization': f'Bearer {token}'},
    params={'event_id': EVENT_ID, 'page': 1, 'per-page': 20})
data = resp.json()
print(f"Total: {data['_meta']['totalCount']}")
for item in data['items']:
    print(f"  #{item['id']}: {item['login_name']} ({item['unique_id']})")

# Step 3: Get single registrant
uid = data['items'][0]['unique_id']
resp = requests.get(f'{BASE_URL}/registrants/{uid}',
    headers={'Authorization': f'Bearer {token}'},
    params={'event_id': EVENT_ID})
registrant = resp.json()['data']
print(f"Name: {registrant['data_fields'].get('first_name', '')} {registrant['data_fields'].get('last_name', '')}")

```

JavaScript (Node.js)

```

const API_KEY = 'qss_6cfb54cbf59ec6a14f9553a3af989e70';
const API_SECRET = '23d4551385b837db667cf8e65108d51507d0f45a';
const BASE_URL = 'https://qss.qestsoln.com/api/v1';

```

```

const EVENT_ID = 'AB6E9834-5752-2A1A-2CF9-58391339841B';

// Step 1: Get Bearer Token
const authResp = await fetch(`${BASE_URL}/oauth/token`, {
  method: 'POST',
  headers: {
    'Authorization': `Basic ***`      'Content-Type': 'application/x-www-form-urlencoded',
  },
  body: 'grant_type=client_credentials',
});
const { access_token: token } = await authResp.json();

// Step 2: List registrants
const listResp = await fetch(
  `${BASE_URL}/registrants?event_id=${EVENT_ID}&page=1&per-page=20`,
  { headers: { 'Authorization': `Bearer ${token}` } }
);
const list = await listResp.json();
console.log(`Total: ${list._meta.totalCount}`);

// Step 3: Get single registrant
const uid = list.items[0].unique_id;
const detailResp = await fetch(
  `${BASE_URL}/registrants/${uid}?event_id=${EVENT_ID}`,
  { headers: { 'Authorization': `Bearer ${token}` } }
);
const detail = await detailResp.json();
console.log(detail.data.data_fields);

```

cURL (Quick Test)

```

# Set your credentials
API_KEY="qss_6cfb54cbf59ec6a14f9553a3af989e70"
API_SECRET="23d4551385b837db667cf8e65108d51507d0f45a"
EVENT_ID="AB6E9834-5752-2A1A-2CF9-58391339841B"

# Step 1: Get token
BASIC=$(echo -n "${API_KEY}:${API_SECRET}" | base64)
TOKEN=$(curl -s -X POST https://qss.qestsoln.com/api/v1/oauth/token \
  -H "Authorization: Basic ***" \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -d "grant_type=client_credentials" | python3 -c "import sys,json;print(json.loads(sys.stdin.read())['access_token'])")

# Step 2: List registrants
curl -s "https://qss.qestsoln.com/api/v1/registrants?event_id=${EVENT_ID}&page=1" \
  -H "Authorization: Bearer $TOKEN"

```

```
-H "Authorization: Bearer ${TOKEN}" | python3 -m json.tool

# Step 3: Get single registrant
curl -s "https://qss.qestsoln.com/api/v1/registrants/DB63B506-3536-B771-018B-25"
-H "Authorization: Bearer ${TOKEN}" | python3 -m json.tool
```

Quick Reference

Endpoint	Method	Auth	Description
/api/v1/oauth/token	POST	Basic	Exchange API key/secret for Bearer token
/api/v1/registrants	GET	Bearer	List registrants (paginated)
/api/v1/registrants/{unique_id}	GET	Bearer	Get single registrant details

FAQ

- Q: How long does the Bearer Token last?** A: 30 minutes. After expiry, request a new token using the same OAuth endpoint.
- Q: Can I access data from multiple events?** A: No. Each exhibitor account is tied to one event. You can only query data within your assigned event.
- Q: What happens if I exceed the rate limit?** A: You'll receive a 429 Too Many Requests response. Wait for the window to reset (check the X-Rate-Limit-Reset header for the remaining time in seconds).

Q: What if my API credentials are compromised? A: Contact your QSS administrator immediately to revoke or regenerate your credentials.